

JS Typecasting

1. What is Typecasting?

- a. *Typecasting* is the process of converting one data type into another.
- b. It helps ensure that values are in the right format when performing operations.
- c. This can involve converting a string into a number, a number into a boolean, or vice versa.
- d. Typecasting can occur *implicitly* (*automatic conversion by the JavaScript engine*) or *explicitly* (*manual conversion by the programmer*).

2. What is the need to perform typecasting?

- a. We perform typecasting to:
 - i. **Match data types** before operations (e.g., adding numbers instead of strings).
 - ii. **Avoid unexpected results** due to JavaScript's automatic type conversion.
 - iii. **Ensure compatibility** when passing values between functions or APIs.
 - iv. **Format data** for display purposes (e.g., converting numbers to strings).
 - v. **For Example:**
let num = "10";
let result = num + 5; // "105" (string concatenation, not addition)

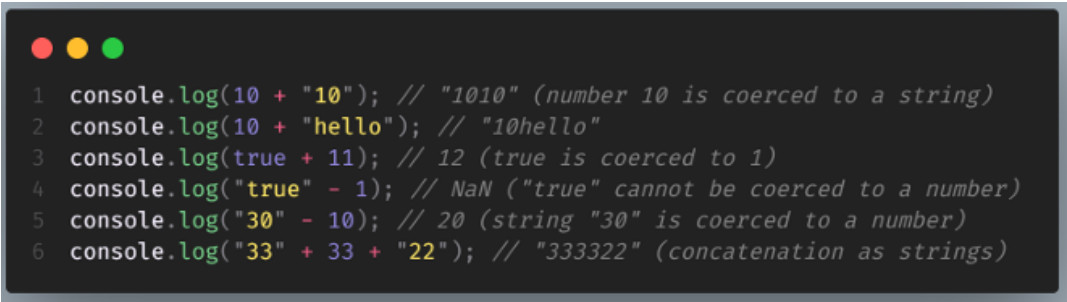
- vi. If we want numerical addition:
let result = Number(num) + 5; // 15

3. Types of Typecasting:

- a. There are mainly two types of typecasting:
 - i. Implicit Typecasting (Type Coercion)
 - ii. Explicit Typecasting

4. Implicit Typecasting (*Type Coercion*):

- a. The JavaScript engine *automatically* converts data types during operations.
- b. JavaScript converts the data type based on the operation being performed.
- c. For Example:



```
1 console.log(10 + "10"); // "1010" (number 10 is coerced to a string)
2 console.log(10 + "hello"); // "10hello"
3 console.log(true + 11); // 12 (true is coerced to 1)
4 console.log("true" - 1); // NaN ("true" cannot be coerced to a number)
5 console.log("30" - 10); // 20 (string "30" is coerced to a number)
6 console.log("33" + 33 + "22"); // "333322" (concatenation as strings)
```

5. Explicit Typecasting:

- a. The developer explicitly converts one data type to another using built-in functions like *Number()*, *String()*, or *Boolean()*.
- b. Done manually by the programmer.

c. For Example:

```
1 console.log(10 + Number("10")); // 20 (explicitly converting string "10" to number)
2 console.log(10 + Number("a")); // NaN ("a" cannot be converted to a number)
3 console.log(String(true) + 10); // "true10" (true converted to string)
4 console.log(Boolean("a") + 11); // 12 (non-empty string is coerced to true, true = 1)
5 console.log(Boolean("") + true); // 1 (empty string is false, false + true = 1)
```

d. String Concatenation vs. Arithmetic:

```
1 console.log("33" - "33"); // 0 (both strings are coerced to numbers)
2 console.log("33" + 33 + "22"); // "333322" (concatenation as strings)
3 console.log("33" - 33 + "33"); // "033" (first coerced to number, then concatenated)
```

e. Boolean Typecasting:

```
1 console.log(Boolean("")); // false (empty string is falsy)
2 console.log(Boolean(" ")); // true (non-empty string is truthy)
3 console.log(Boolean(null)); // false (null is falsy)
4 console.log(Boolean("-10") - 10); // -9 (non-empty string "-10" is truthy)
```

f. String to Number Conversion:

```
1 console.log("String" - 10); // NaN (string cannot be converted to a number)
2 console.log("10" - 10); // 0 (string "10" is coerced to number)
3 console.log("a" - 33); // NaN ("a" cannot be coerced to a number)
```

g. Combining Types:

```
1 console.log(33 - "33" + Boolean("") + "hello"); // "0hello"
2 console.log(Boolean(String(null)) - 10); // -9
```